

CS 4341 Assignment 1

Due 2026-09-09

1 Overview

In this assignment, you will implement A* graph search and Learning Real-Time A* (LRTA*) for the puzzle game Rune Shifter, and design a heuristic for each search algorithm. You will also explain and evaluate your work in a video code walk-through.

2 Rune Shifter

Rune Shifter is played on a five-by-five grid. A move shifts all the pieces in one of four directions. Matching runes merge when they collide, and the resulting merged rune no longer moves. The goal is to merge every rune.

The provided `runeshifter.py` file defines the game state, legal actions, transition function, action costs, and goal test. Do not duplicate the game rules in your search implementation.

3 Starter Files

Download and extract the starter bundle. Keep all of its files together in the same directory while working. The files are organized as described below.

3.1 `main.py`

Complete these four functions in `main.py`:

1. `astar_heuristic(state) -> float`
2. `astar_search(problem, h) -> SearchResult | None`
3. `lrtastar_heuristic(state) -> float`
4. `lrtastar_search(problem, h, max_steps) -> SearchResult | None`

Both heuristic functions must accept a valid Rune Shifter state and return a Python `float`. The search functions must use the heuristic passed as `h`. This allows the search algorithms to be tested independently of your own heuristics.

A* must be implemented as graph search. When frontier priorities tie, remove the node that was added first. Iterate over actions in the order supplied by `problem.actions`; do not sort or reorder them.

For A*, `expanded` counts each node removed from the frontier for processing, including the goal node.

LRTA* must stop and return `None` if it has executed `max_steps` actions without reaching a goal or if a non-goal state has no available actions. When estimated action costs tie, select the first action supplied by `problem.actions`.

For LRTA*, `expanded` counts every state processed, including the initial state, repeated states, and the final goal state.

You must design a heuristic for each search algorithm. Although the public tests only verify that your heuristics can be called and return Python `float` values, heuristic quality will affect leaderboard performance and your video-recording grade.

3.2 `runeshifter.py`

The provided `runeshifter.py` file implements Rune Shifter's `GameState`, actions, transition function, action costs, and goal test. Use its `RuneShifter` problem through the supplied search interface; do not copy the game rules into `main.py` or modify this starter file.

3.3 `search.py`

The provided `search.py` file contains the interfaces and types used by the assignment:

- `SearchProblem` provides:
 - `initial`;
 - `actions(state)`;
 - `result(state, action)`;
 - `action_cost(state, action, next_state)`; and
 - `is_goal(state)`.
- `Heuristic` is a callable that accepts a state and returns a `float`.
- `SearchResult` contains:
 - `actions`, the action sequence from the initial state to a goal; and
 - `expanded`, the expanded-node count.

Use the provided interfaces and types directly. Do not modify their required behavior or duplicate their definitions in `main.py`.

3.4 `test_heuristics.py` and `test_cases.py`

The provided `test_heuristics.py` and `test_cases.py` files contain the reference heuristics and public test cases.

From the assignment directory, run:

```
python -m pytest -q test_cases.py
```

The public tests compare both search functions against precomputed expected results on fixed Rune Shifter states. They pass the supplied grading heuristics from `test_heuristics.py`, rather than your heuristics, to the search functions.

After submission, Gradescope will compare your search behavior on additional private states. Each search runs with a timeout, so a nonterminating implementation fails. Passing the public tests alone does not guarantee full correctness.

4 Leaderboard

Submissions that pass all automated tests will be evaluated on ten fixed leaderboard states. Both search algorithms will run on every state using your own corresponding heuristic.

Gradescope reports three leaderboard columns:

- **A* Path Cost:** total path cost across the ten A* searches;
- **A* Expanded Nodes:** total expanded nodes across the ten A* searches; and
- **LRTA* Path Cost:** total path cost across the ten LRTA* searches.

Lower values are better for every leaderboard column. Prizes will be awarded for the best A* heuristic (based on path cost with expanded nodes as the tie breaker) and LRTA* heuristic based on path cost.

5 Submission

This is a group project for 2–4 students. Form your group in the submission system before uploading either deliverable. Only one group member should submit the following on behalf of the group:

1. upload your completed code, named exactly `main.py`; and
2. upload your video recording as one `.mp4` file.

Upload the files separately, not in a ZIP archive. Do not rename `main.py`, any required function, or any required function signature. Only `main.py` is graded as code; video points are recorded separately.

6 Video Recording

Submit one continuous, unedited recording with your camera on and screen shared. Treat the entire recording as a live code walk-through: answer while showing the relevant code, searches, tests, output, proofs, calculations, or diagrams. Unsupported verbal claims or a separate spoken report do not meet the requirements.

1. Frame and verify the assignment.
 - Explain the problem, required outputs, difference between A* and LRTA*, and your overall strategy.
 - Run the public tests, then identify at least one Rune Shifter state on which you will demonstrate both searches.
2. Demonstrate, explain, and evaluate A*.
 - In `main.py`, trace the frontier, costs, repeated-state handling, tie-breaking, and path reconstruction. Point out use of `h` and the problem interface, plus goal and failure handling, action ordering, and `expanded` accounting.
 - Explain and justify your heuristic, then prove or disprove its admissibility and consistency.
 - On the same state or states, run your A* and a simpler global baseline, such as breadth-first search or Dijkstra/uniform-cost search. A* with $h = 0$ is a convenient uniform-cost baseline.
 - Show and interpret path cost, expanded nodes, and running time. Use the comparison to evaluate correctness, solution quality, search effort, heuristic effectiveness, and design tradeoffs; standalone metrics are insufficient.
3. Demonstrate, explain, and evaluate LRTA*.
 - In `main.py`, trace action selection, learned-value updates, tie-breaking, and `max_steps`. Point out use of `h` and the problem interface, plus goal and failure handling, action ordering, and `expanded` accounting.
 - Explain and justify your heuristic, including any difference from the A* heuristic, then prove or disprove its admissibility and consistency.
 - On the same state or states, run your LRTA* and a simpler local baseline, such as non-learning greedy/local search or beam search.
 - Show and interpret path cost, expanded nodes, repeated-state behavior, and running time. Use the comparison to evaluate correctness, solution quality, search effort, heuristic effectiveness, and design tradeoffs; standalone metrics are insufficient.

Your explanation must match the submitted code.

7 Grading

7.1 Automated Test Cases: 40%

The 20 graded test cases are worth 2% each. Your submission must be named `main.py`, and only that file can be submitted for code grading.

7.2 Video Recording: 60%

The video recording is worth 60% and is divided into timing and technical-content components.

Time Limit: 20%

- One continuous, unedited recording of 10 minutes or less: 20%.
- More than 10 minutes or an edited recording: 0%.

Any cut, splice, speed change, or other alteration of the recording forfeits these 20 points.

Technical Content: 40%

Technical-content points require a correct implementation, a matching explanation, and the on-screen evidence specified above.

- **Framing and verification: 4%.** Complete the framing and public-test demonstration above.
- **A* demonstration, design, and analysis: 18%.** Complete the A* code walk-through, heuristic analysis, live baseline comparison, and evidence-based evaluation above.
- **LRTA* demonstration, design, and analysis: 18%.** Complete the LRTA* code walk-through, heuristic analysis, live baseline comparison, and evidence-based evaluation above.